

What will be the output of this recursive function?

```
def rec👉 :
```

```
    if n <= 0:
```

```
        return 0
```

```
    return n + rec(n-1)
```

```
print(rec(4))
```

- ☐ a. 4
- ☒ b. 10
- ☐ c. Error
- ☐ d. 0

[Clear my choice](#)

Consider a stack with push(A), push(B), pop(), push(C), pop(), pop(). What is the sequence of popped elements?

- ☐ a. A, C, B
- ☐ b. A, B, C
- ☒ c. B, C, A
- ☐ d. B, C, (stack underflow)

[Clear my choice](#)

In Python, which method is used to initialize new objects?

- ☐ a. `__new__`
- ☐ b. `__create__`
- ☐ c. `__start__`
- ☐ d. `__init__`

Next page

Which linked list is best suited for implementing a deque (double-ended queue)?

- ☐ a. Circular singly linked list
- ☐ b. Singly linked list
- ☐ c. Doubly linked list
- ☐ d. Circular doubly linked list

The expression tree (binary tree for arithmetic expression) is best evaluated using:

- ☐ a. Queue
- ☐ b. Stack
- ☐ c. Priority Queue
- ☐ d. Linked List

What will be the output of this recursive function?

```
def rec👉 :
```

```
    if n <= 0:
```

```
        return 0
```

```
    return n + rec(n-1)
```

```
print(rec(4))
```

Time left 0:58:28

- ☐ a. 4
- ☐ b. 10
- ☐ c. Error
- ☐ d. 0

After executing the following Python code:

```
stack = []
```

```
stack.append(1)
```

```
stack.append(2)
```

```
stack.pop()
```

```
stack.append(3)
```

```
print(stack)
```

What will be the output?

- ☐ a. [2, 3]
- ☐ b. [1, 3]
- ☐ c. [1, 2, 3]
- ☐ d. [3, 2, 1]

Suppose a singly linked list has n nodes. What is the time complexity to delete the last node if only the head pointer is given?

- ☐ a. $O(1)$
- ☐ b. $O(\log n)$
- ☐ c. $O(1)$
- ☐ d. $O(n^2)$

Which linked list allows traversal in both directions?

- ☐ a. None
 - ☐ b. Doubly linked list
 - ☐ c. Singly linked list
 - ☐ d. Circular singly linked list
-

If a linked list has 6 nodes, how many NULL pointers does it contain (for singly linked list)?

- ☐ a. 6
- ☐ b. 2
- ☐ c. 1
- ☐ d. 5

Which operation is faster in the linked list compared to arrays?

- ☐ a. Traversal
- ☐ b. Insertion/deletion at beginning
- ☐ c. Random access
- ☐ d. Binary search

What will the following code print?

```
a = (1, 2, 3)
```

```
a[0] = 99
```

```
print(a)
```

- ☐ a. **None**
- ☐ b. **Error (tuple is immutable)**
- ☐ c. **(99, 2, 3)**
- ☐ d. **(1, 2, 3)**

Deletion from the end of a singly linked list (head pointer only) requires:

- ☐ a. $O(\text{👎})$
- ☐ b. $O(1)$
- ☐ c. $O(n \log n)$
- ☐ d. $O(n^2)$

Which of the following represents a recursive data structure?

- ☐ a. **Linked list**
- ☐ b. **Dictionary**
- ☐ c. **Tuple**
- ☐ d. **List**

After the operations:

```
stack = []
```

```
stack.append(1)
```

```
stack.pop()
```

```
stack.pop()
```

What happens?

- ☐ a. Returns -1
- ☐ b. Raises IndexError
- ☐ c. Prints None
- ☐ d. Stack is empty, no error

Which pointer helps in backward traversal?

- ☐ a. **next**
- ☐ b. **head**
- ☐ c. **tail**
- ☐ d. **prev**

When inserting a node between two nodes in a doubly linked list, how many pointer updates are required?

- ☐ a. 5
- ☐ b. 4
- ☐ c. 3
- ☐ d. 2

Compared to singly linked list, which operation is faster in a doubly linked list?

- ☐ a. Searching
- ☐ b. Deletion of a specific node
- ☐ c. Sorting
- ☐ d. Traversal

The best data structure to check if parentheses are balanced is a

- ☐ a. Stack
- ☐ b. Tree
- ☐ c. List
- ☐ d. Queue

Pushing onto a stack that is already full results in:

- ☐ a. **Overflow**
- ☐ b. **Crash**
- ☐ c. **User flow**
- ☐ d. **Underflow**

The minimum number of stacks needed to implement a queue is

- ☐ a. 3
- ☐ b. 1
- ☐ c. 2
- ☐ d. 4

A queue implemented using a non-circular singly linked list, enqueue at head, dequeue from tail. The time complexities are:

- ☐ a. $\Theta(n)$, $\Theta(n)$
- ☐ b. $\Theta(n)$, $\Theta(1)$
- ☐ c. $\Theta(1)$, $\Theta(n)$
- ☐ d. $\Theta(1)$, $\Theta(1)$

Consider a stack with push(A), push(B), pop(), push(C), pop(), pop(). What is the sequence of popped elements?

- ☐ a. A, C, B
- ☐ b. A, B, C
- ☐ c. B, C, A
- ☐ d. B, C, (stack underflow)

A priority queue is best implemented using:

- ☐ a. Array only
- ☐ b. Heap
- ☐ c. Stack
- ☐ d. Linked List

Which application most commonly uses a queue?

- ☐ a. Infix to postfix conversion
- ☐ b. Function call recursion
- ☐ c. CPU task scheduling
- ☐ d. Undo operation in text editor

Which application most commonly uses a queue?

- ☐ a. Infix to postfix conversion
- ☐ b. Function call recursion
- ☒ c. CPU task scheduling
- ☐ d. Undo operation in text editor

[Clear my choice](#)

A priority queue is best implemented using:

- ☐ a. Array only
- ☒ b. Heap
- ☐ c. Stack
- ☐ d. Linked List

[Clear my choice](#)

Given a stack with push(), pop(), and empty() operations, The task is to delete the middle element of it without using any additional data structure.

Input: s = [5, 8, 6, 7, 6, 6, 5, 10, 12, 9]

Output: [9, 12, 10, 5, 6, 7, 6, 8, 5]



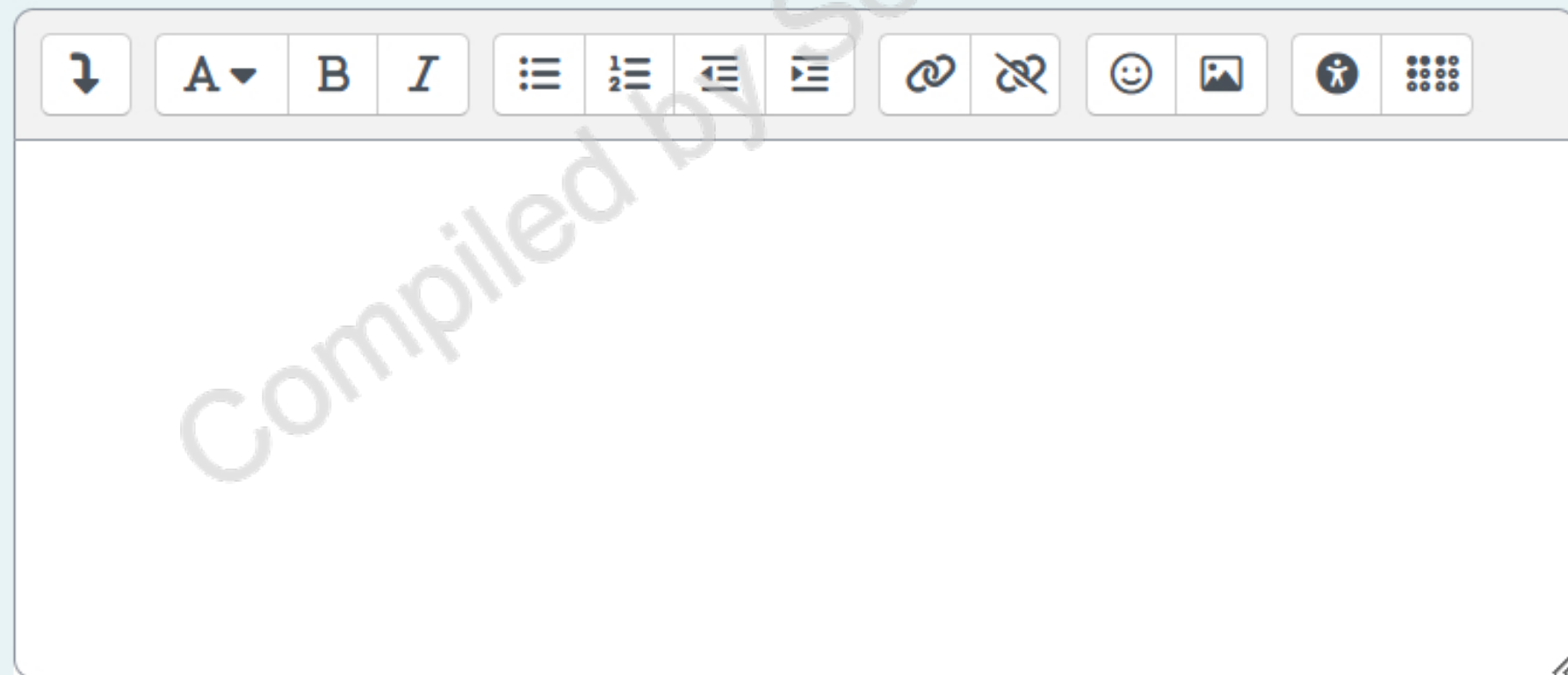
A rich text editor toolbar with various icons for text formatting, alignment, and insertion. The icons include: a downward arrow, bold (A), italic (I), bulleted list, numbered list, decrease indent, increase indent, link, unlink, smiley face, image, person, and a grid of dots. Below the toolbar is a large, empty text area with a blue border.

Given an integer k and a queue of integers, The task is to reverse the order of the first k elements of the queue, leaving the other elements in the same relative order.

Input: $q = 1\ 2\ 3\ 4\ 5$, $k = 3$

Output: $3\ 2\ 1\ 4\ 5$

Explanation: After reversing the first 3 elements from the given queue the resultant queue will be $3\ 2\ 1\ 4\ 5$.



The image shows a rich text editor toolbar with the following icons from left to right: a downward arrow, a bold 'A' with a dropdown arrow, an italic 'B', a slanted 'I', a bulleted list icon, a numbered list icon, a left-align icon, a right-align icon, a link icon, a broken link icon, a smiley face icon, a picture icon, a person icon, and a grid icon. Below the toolbar is a large, empty text area.

Implement a circular queue of size N.

Perform operations:

Time left 0:39:34

Enqueue

Dequeue

Display queue elements

The queue should reuse empty spaces created after deletions.



A rich text editor toolbar with the following icons from left to right: undo (curved arrow), bold (A with a downward arrow), italic (I), bulleted list (three horizontal lines with dots), numbered list (three horizontal lines with numbers 1, 2, 3), decrease indent (left arrow with three horizontal lines), increase indent (right arrow with three horizontal lines), link (chain link), unlink (chain link with a slash), smiley (happy face), image (picture icon), user (person icon), and grid (3x3 dots). Below the toolbar is a large empty text area for editing.

You are given the head of a linked list, You have to return the value of the middle node of the linked list.

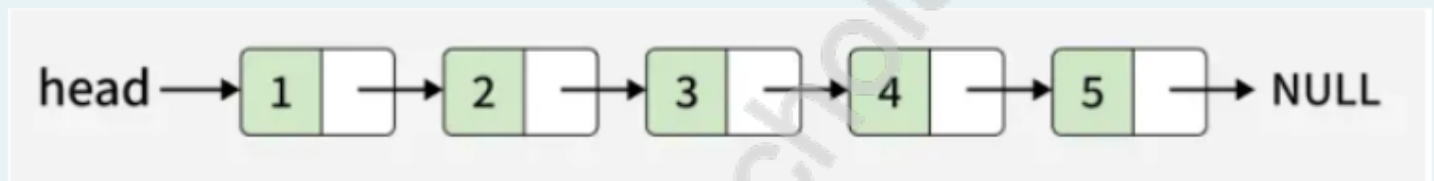
Time left 0:38:32

If the number of nodes is odd, return the middle node value.

If the number of nodes is even, there are two middle nodes, so return the second middle node value.

Examples1:

Input:



Output: 3

Explanation: The given linked list is 1->2->3->4->5 and its middle is 3.

Constraints:

$1 \leq \text{no. of nodes} \leq 10^5$

$1 \leq \text{node} \rightarrow \text{data} \leq 10^5$

Given two polynomial numbers represented by a linked list. The task is to add these lists meaning the coefficients with the same variable powers.

Note: Given polynomials are sorted in decreasing order of power.

Time left 0:37:27

Examples:

Input: LinkedList1: x^2 , LinkedList2: x^3

Output: $x^3 + x^2$

Explanation: Since, x^2 and x^3 both have different powers as 2 and 3. So, their coefficient can't be added up.

Input: LinkedList1: $x^3 \rightarrow 2x^2$, LinkedList2: $3x^3 \rightarrow 4x^2$

Output: $4x^3 \rightarrow 6x^2$

Explanation: Since, x^3 has two different coefficients as 3 and 1. Adding them up will lead to $4x^3$. Also, x^2 has two coefficients as 4 and 2. So, adding them up will give $6x^2$.

Note: Try to solve the question without using any extra space.

Expected Time Complexity: $O(n+m)$

Expected Auxiliary Space: $O(1)$

Constraints:

$1 \leq \text{no. of nodes LinkedList1, LinkedList2} \leq 10^5$

$1 \leq \text{node} \rightarrow \text{data} \leq 10^6$

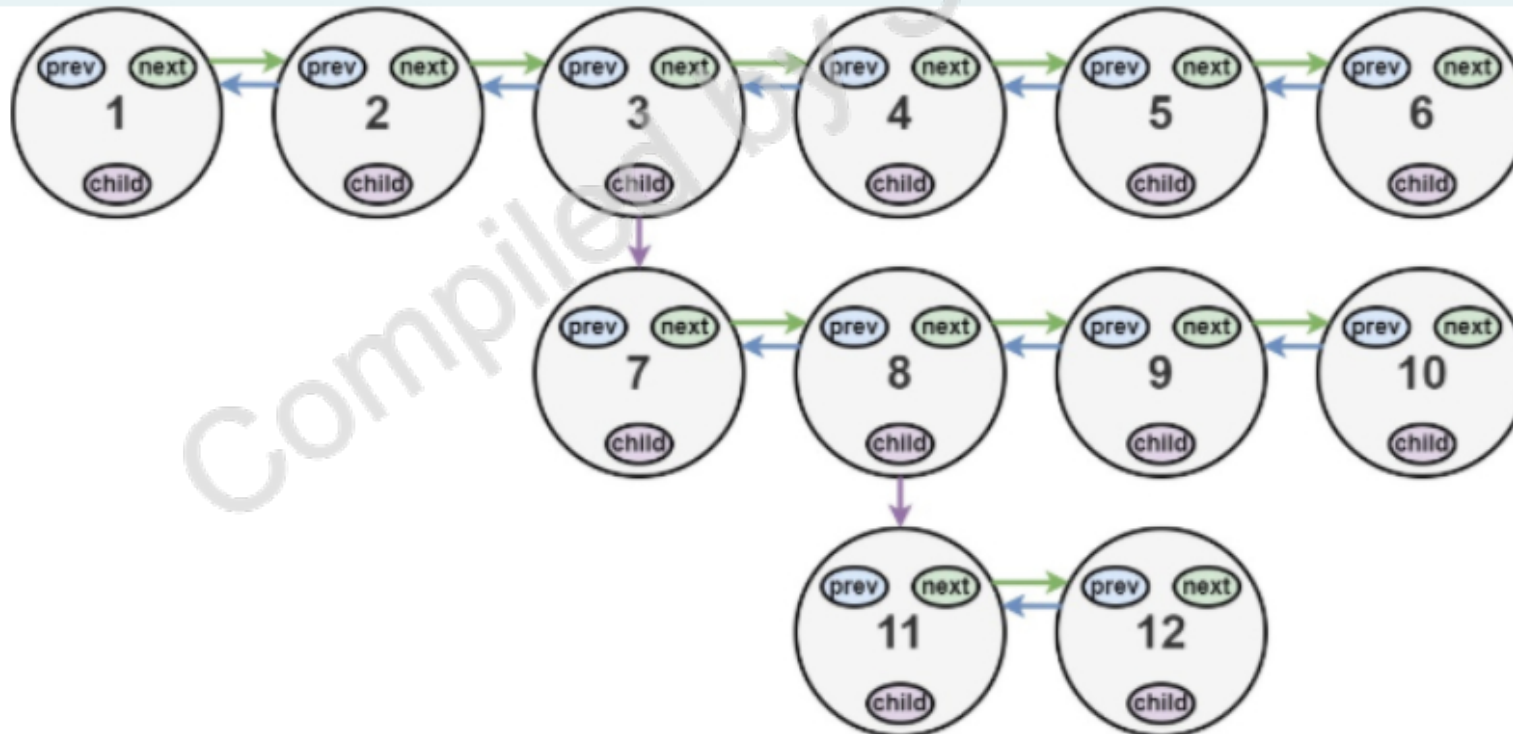
You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Time left 0:34:56

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let `curr` be a node with a child list. The nodes in the child list should appear after `curr` and before `curr.next` in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:



Given an array `nums`, you can perform the following operation any number of times:

Select the adjacent pair with the minimum sum in `nums`. If multiple such pairs exist, choose the leftmost pair.

Time left 0:36:01

Replace the pair with their sum.

Return the minimum number of operations needed to make the array non-decreasing.

An array is said to be non-decreasing if each element is greater than or equal to its previous element (if it exists).

Example 1:

Input: `nums = [5,2,3,1]`

Output: 2

Explanation:

The pair (3,1) has the minimum sum of 4. After replacement, `nums = [5,2,4]`.

The pair (2,4) has the minimum sum of 6. After replacement, `nums = [5,6]`.

The array `nums` became non-decreasing in two operations.

Example 2:

Input: `nums = [1,2,2]`

Output: 0

Explanation: The array `nums` is already sorted.

Constraints:

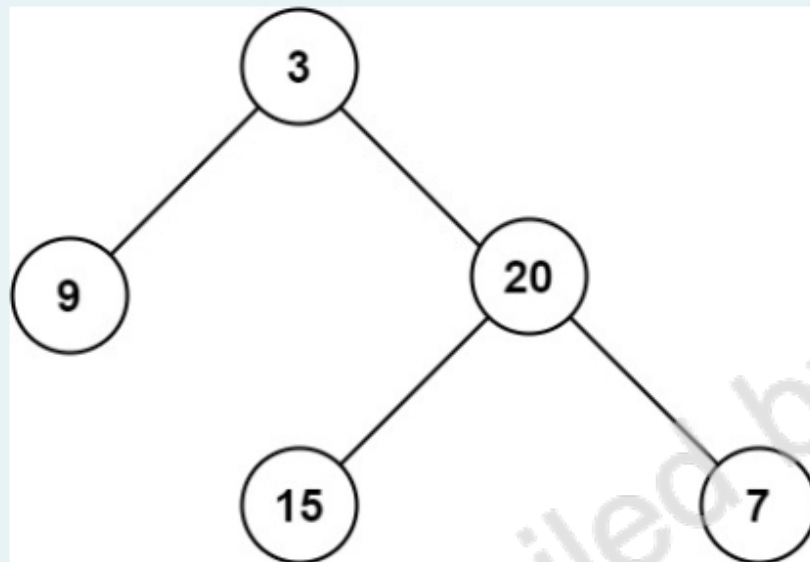
$1 \leq \text{nums.length} \leq 50$

$-1000 \leq \text{nums}[i] \leq 1000$

1. Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: 3

Example 2:

Input: root = [1,null,2]

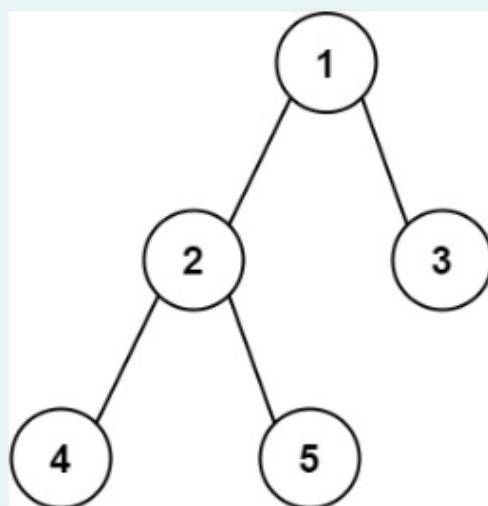
Output: 2

1. Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

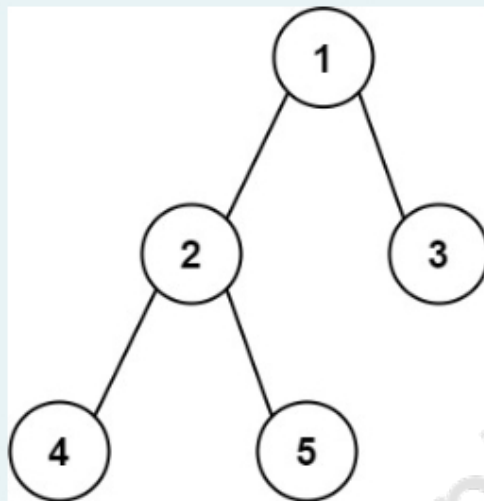
Output: 1

1. Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

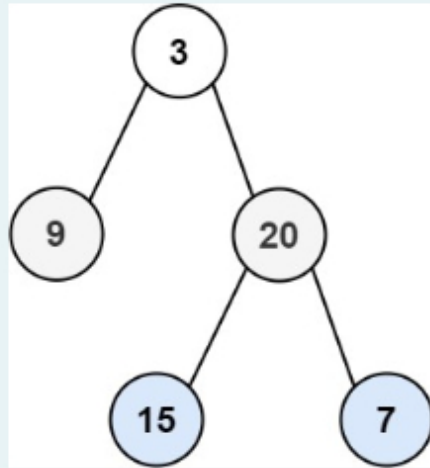
Example 2:

Input: root = [1,2]

Output: 1

1. Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]

Output: [[1]]

Example 3:

Input: root = []

Output: []

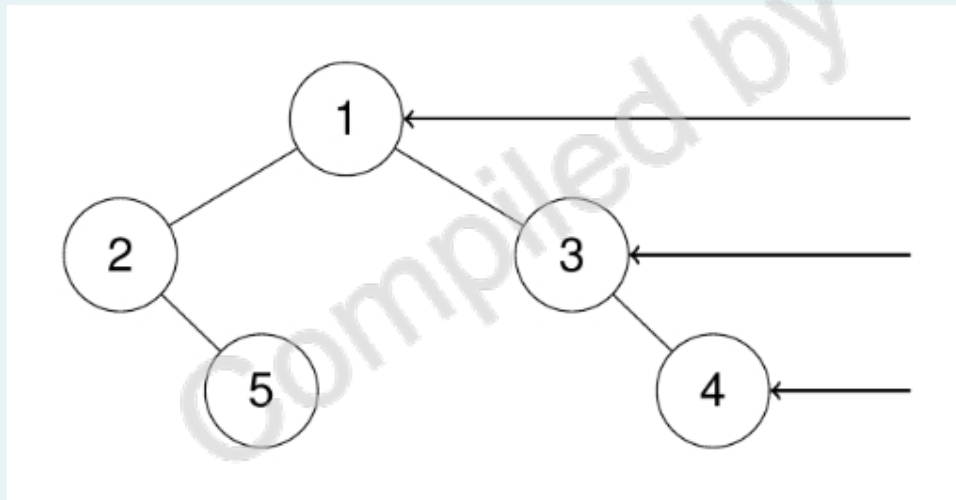
1. Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

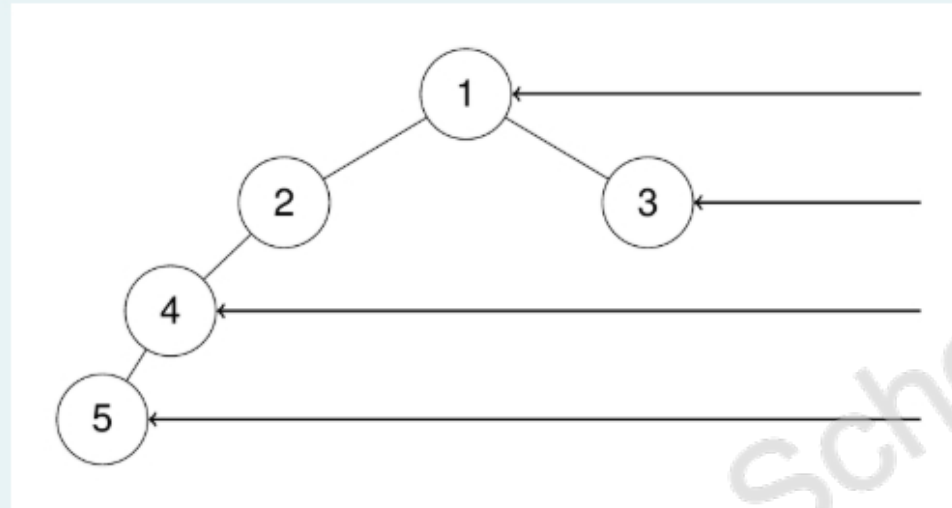


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

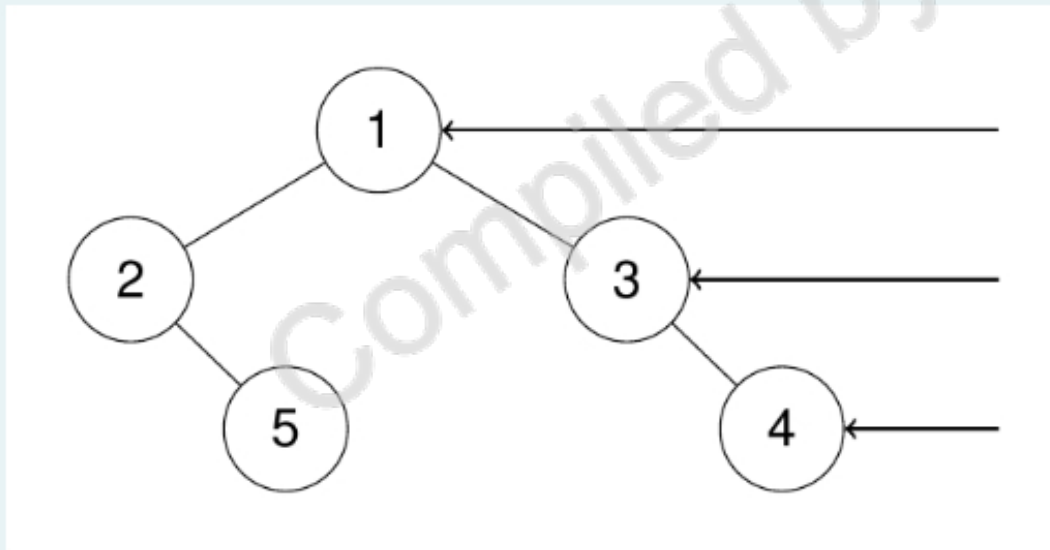
1. Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

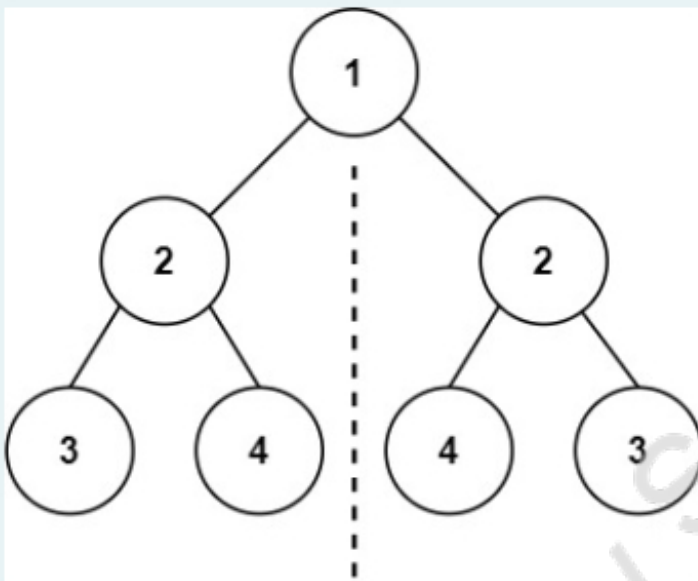
Explanation:



Example 2:

1. Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetrical or not).

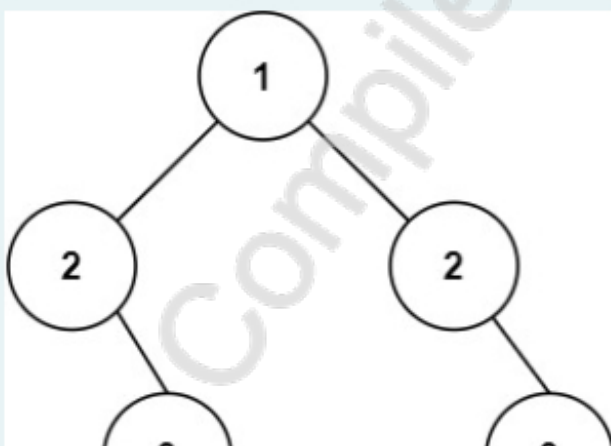
Example 1:



Input: root = [1,2,2,3,4,4,3]

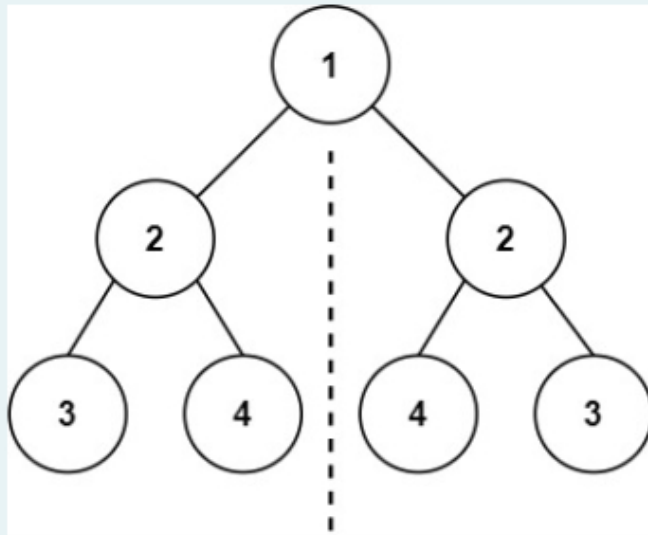
Output: true

Example 2:



1. Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

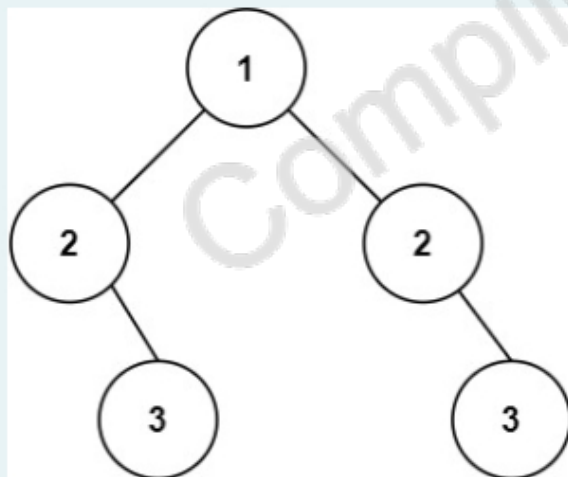
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

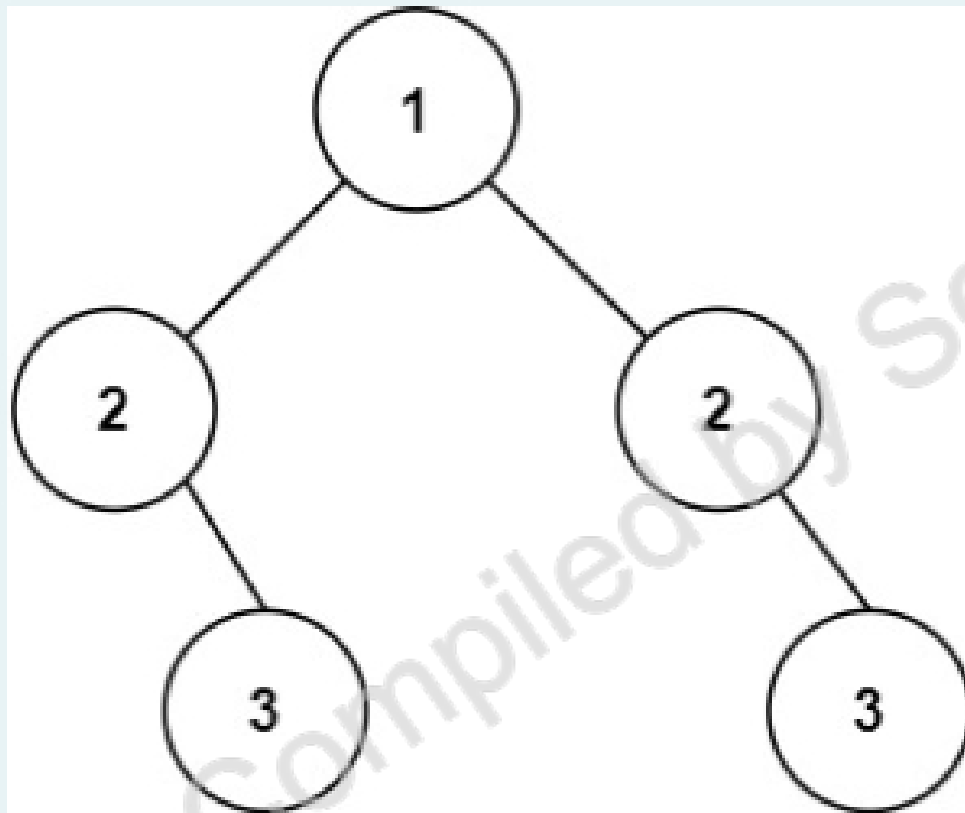
Example 2:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



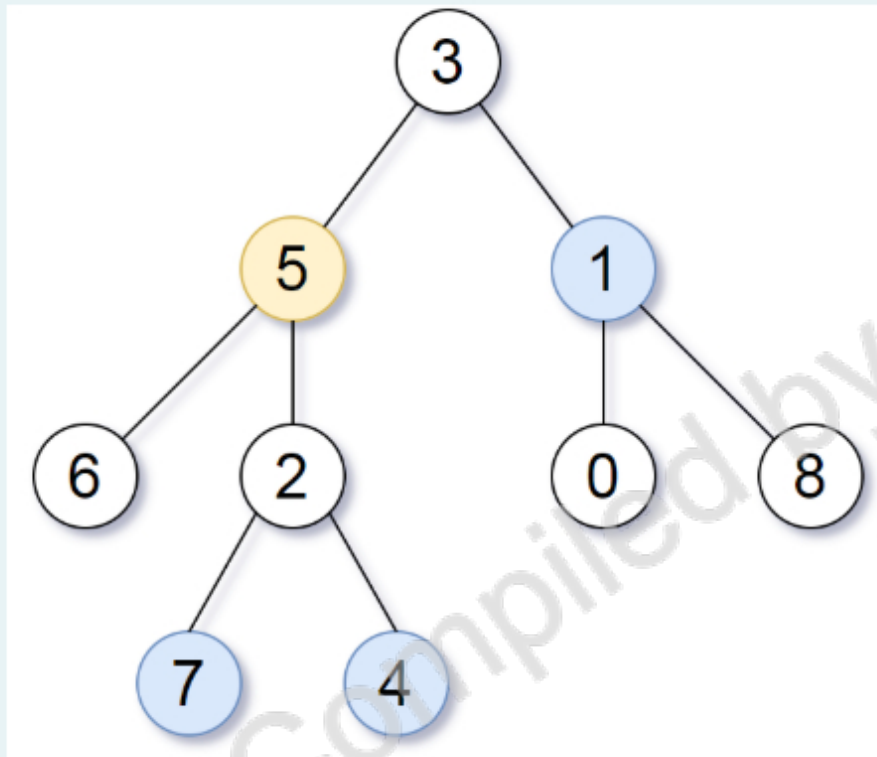
Input: root = [1,2,2,null,3,null,3]

Output: false

1. Given the root of a binary tree, the value of a target node `target`, and an integer `k`, return an array of the values of all nodes that have a distance `k` from the target node.

You can return the answer in any order.

Example 1:

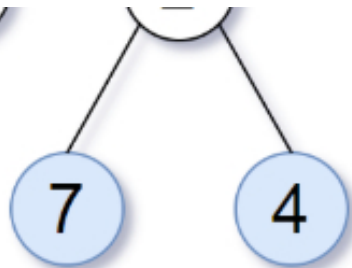


Input: `root = [3,5,1,6,2,0,8,null,null,7,4]`, `target = 5`, `k = 2`

Output: `[7,4,1]`

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

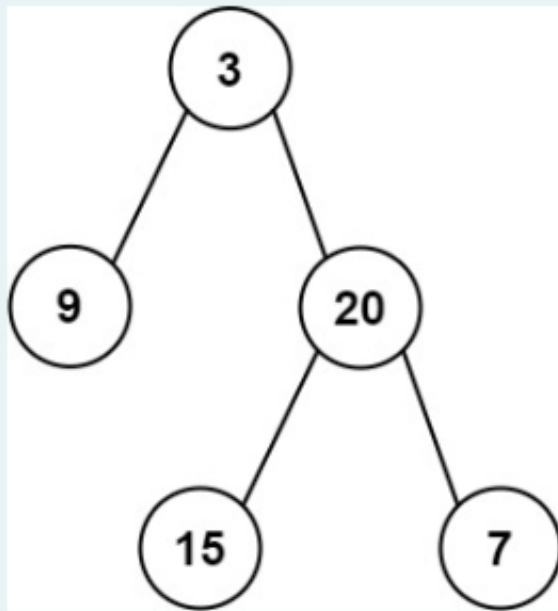
Example 2:

Input: root = [1], target = 1, k = 3

Output: []

1. Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]

Output: [3,9,20,null,null,15,7]

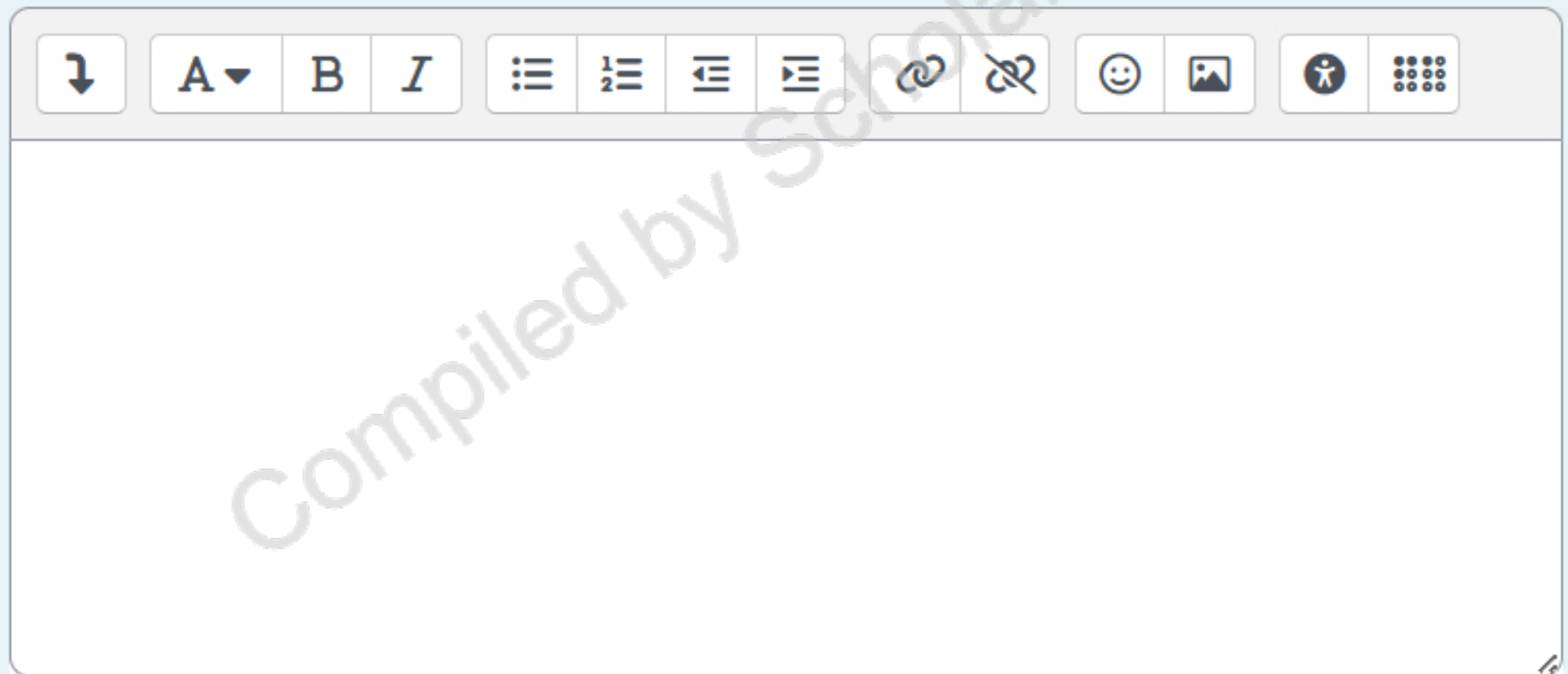
Example 2:

Input: inorder = [-1], postorder = [-1]

Output: [-1]

Write a Python program for Quick Sort using:

- Last element as pivot
- Random pivot



The image shows a code editor window. The toolbar at the top contains the following icons from left to right: a cursor icon, a dropdown menu with 'A' and a downward arrow, a bold 'B' icon, an italic 'I' icon, a list of three horizontal lines, a list of three horizontal lines with a subscript '2', a left-align icon, a right-align icon, an undo icon, a redo icon, a smiley face icon, a picture icon, a person icon, and a grid of nine dots icon. The main area of the editor is a large, empty white space. A diagonal watermark reading 'Compiled by Scholars' Hub' is visible across the center of the editor area.

You are given student records. Sort them based on marks (descending). If marks are same, sort by name.

Input:

```
students = [  
    ("Ankit", 85),  
    ("Riya", 92),  
    ("Vikas", 85),  
    ("Neha", 95)  
]
```

Output:

```
[  
    ("Neha", 95),  
    ("Riya", 92),  
    ("Ankit", 85),  
    ("Vikas", 85)  
]
```

While constructing MST, detect if adding an edge creates a cycle.

Time left 0:38:04

Input:

edges = [

(0,1),

(1,2),

(2,0)

]

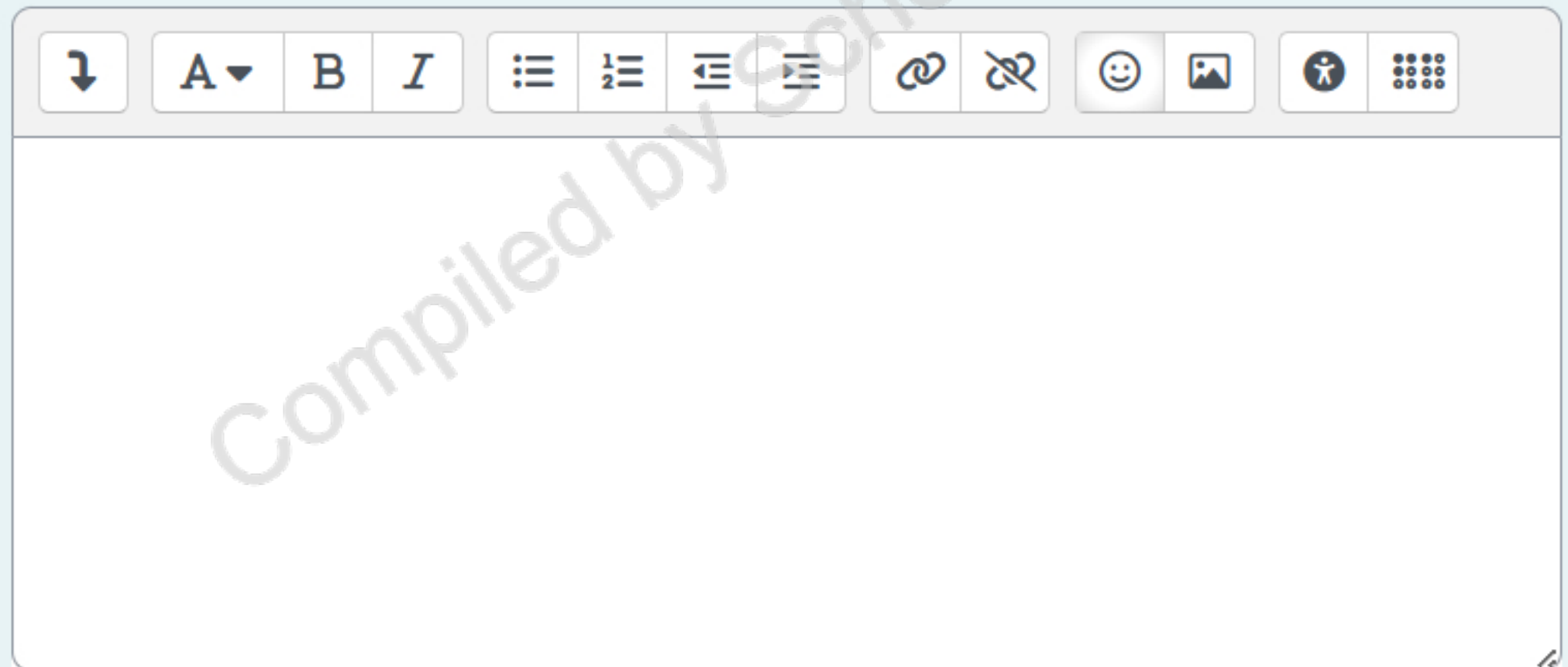
Output: Cycle Detected

Rich text editor toolbar with icons for: Undo, Bold, Italic, Bulleted List, Numbered List, Decrease Indent, Increase Indent, Link, Unlink, Smiley, Image, User, and Grid.

Q7) An e-commerce website wants to sort products based on price. Implement Merge Sort (not built-in sort)

Input: prices = [1200, 450, 999, 230, 1500]

Output: [230, 450, 999, 1200, 1500]



In a full binary tree, number of leaf nodes is:

- ☐ a. $2n$
- ☐ b. $n/2n$
- ☐ c. $(n-1)/2$
- ☒ d. $(n+1)/2$

[Clear my choice](#)

A graph where every pair of vertices is connected:

- ☐ a. Tree
- ☒ b. Complete Graph
- ☐ c. Sparse Graph
- ☐ d. Bipartite Graph

[Clear my choice](#)

Which data structure is used in BFS?

- ☐ a. Heap
- ☒ b. Queue
- ☐ c. Stack
- ☐ d. Tree

[Clear my choice](#)

In an undirected graph, sum of degrees of all vertices is:

- ☐ a. E^2
- ☒ b. $2E$
- ☐ c. E
- ☐ d. V

If a node has no children, it is called:

- ☐ a. Internal node
- ☐ b. Parent node
- ☒ c. Leaf node
- ☐ d. Root

[Clear my choice](#)

If preorder is:

A B D E C F

and inorder is:

D B E A F C

What is postorder?

- ☐ a. **E D B F C A**
- ☒ b. **D E B F C A**
- ☐ c. **D E B C F A**
- ☐ d. **D B E F C A**

In a full binary tree, if number of internal nodes is i , number of leaf nodes is:

- ☒ a. $i+1$
- ☐ b. $i-1$
- ☐ c. $2i$
- ☐ d. i

[Clear my choice](#)

A binary tree with n nodes has how many edges?

- ☐ a. $n+1$
- ☐ b. n
- ☒ c. $n-1$
- ☐ d. $2n$

[Clear my choice](#)

Assume that we use Bubble Sort to sort n distinct elements in ascending order. When does the best case of Bubble Sort occur?

- ☐ a. There is no best case for Bubble Sort. It always takes $O(n*n)$ time
- ☐ b. When elements are not sorted by any order
- ☐ c. When elements are sorted in descending order
- ☒ d. When elements are sorted in ascending order

[Clear my choice](#)

Which of the following is not true about comparison-based sorting algorithms?

- ☐ a. Any comparison based sorting algorithm can be made stable by using position as a criteria when two elements are compared
- ☒ b. Heap Sort is not a comparison based sorting algorithm
- ☐ c. The minimum possible time complexity of a comparison-based sorting algorithm is $O(n \log n)$ for a random input array
- ☐ d. Counting Sort is not a comparison based sorting algorithm

[Clear my choice](#)

Which data structure is used for Level Order Traversal?

- ☐ a. Stack
- ☐ b. Heap
- ☒ c. Queue
- ☐ d. Array

[Clear my choice](#)

Prim's algorithm is best implemented using:

- ☐ a. Queue
- ☒ b. Priority Queue (Min Heap)
- ☐ c. Array
- ☐ d. Stack

Clear my choice

A cycle in a graph can be detected using:

- ☐ a. Stack only
- ☐ b. BFS only
- ☒ c. DFS
- ☐ d. Queue only

[Clear my choice](#)

Time complexity of Kruskal (with sorting):

- ☒ a. $O(E \log E)$
- ☐ b. $O(V)$
- ☐ c. $O(E)$
- ☐ d. $O(V^2)$

[Clear my choice](#)

Which representation is best for sparse graphs?

- ☐ a. None
- ☐ b. Incidence Matrix
- ☐ c. Adjacency Matrix
- ☒ d. Adjacency List

[Clear my choice](#)

A graph where every pair of vertices is connected:

- ☐ a. Tree
- ☐ b. Complete Graph
- ☐ c. Sparse Graph
- ☐ d. Bipartite Graph

Which data structure is used in BFS?

- ☐ a. **Heap**
- ☐ b. **Queue**
- ☐ c. **Stack**
- ☐ d. **Tree**

In an undirected graph, sum of degrees of all vertices is:

- ☐ a. E^2
- ☐ b. $2E$
- ☐ c. E
- ☐ d. V

In a full binary tree, if number of internal nodes is i , number of leaf nodes is:

- ☐ a. $i+1$
- ☐ b. $i-1$
- ☐ c. $2i$
- ☐ d. i

Which algorithm is NOT used for MST?

- ☐ a. Dijkstra
- ☐ b. Kruskal
- ☐ c. Prim
- ☐ d. None

Time complexity of Kruskal (with sorting):

- ☐ a. $O(E \log E)$
- ☐ b. $O(V)$
- ☐ c. $O(E)$
- ☐ d. $O(V^2)$